

University of Pittsburgh

Introduction to Operating Systems
CS 1550

Spring 2022

Sherif Khattab

ksm73@pitt.edu

(Some slides are from **Silberschatz, Galvin and Gagne ©2013)**

Announcements

- Upcoming deadlines:
  - Lab 3: due on 4/1
  - Homework 9: due on 4/4
  - Project 3: due on 4/11

Previous lecture …

- The problem of large page tables

Muddiest Points (1/2)

- How to determine number of bits (2^n) from day 4gb or 4kb
- Calculating the PTE frame number size.
- calculating the number of frames
- userspace page replacement
- For the final top hat question, how did we get 24 in the answer 24*(2^20)? I understand that we did log(2^20) to get 20, but where did the extra 4 come from?
- I don't understand the last two examples about calculating the PTE s and frames, what information (values) do we need to look at in terms of different questions?
- Could you list the formulas for page table entries?

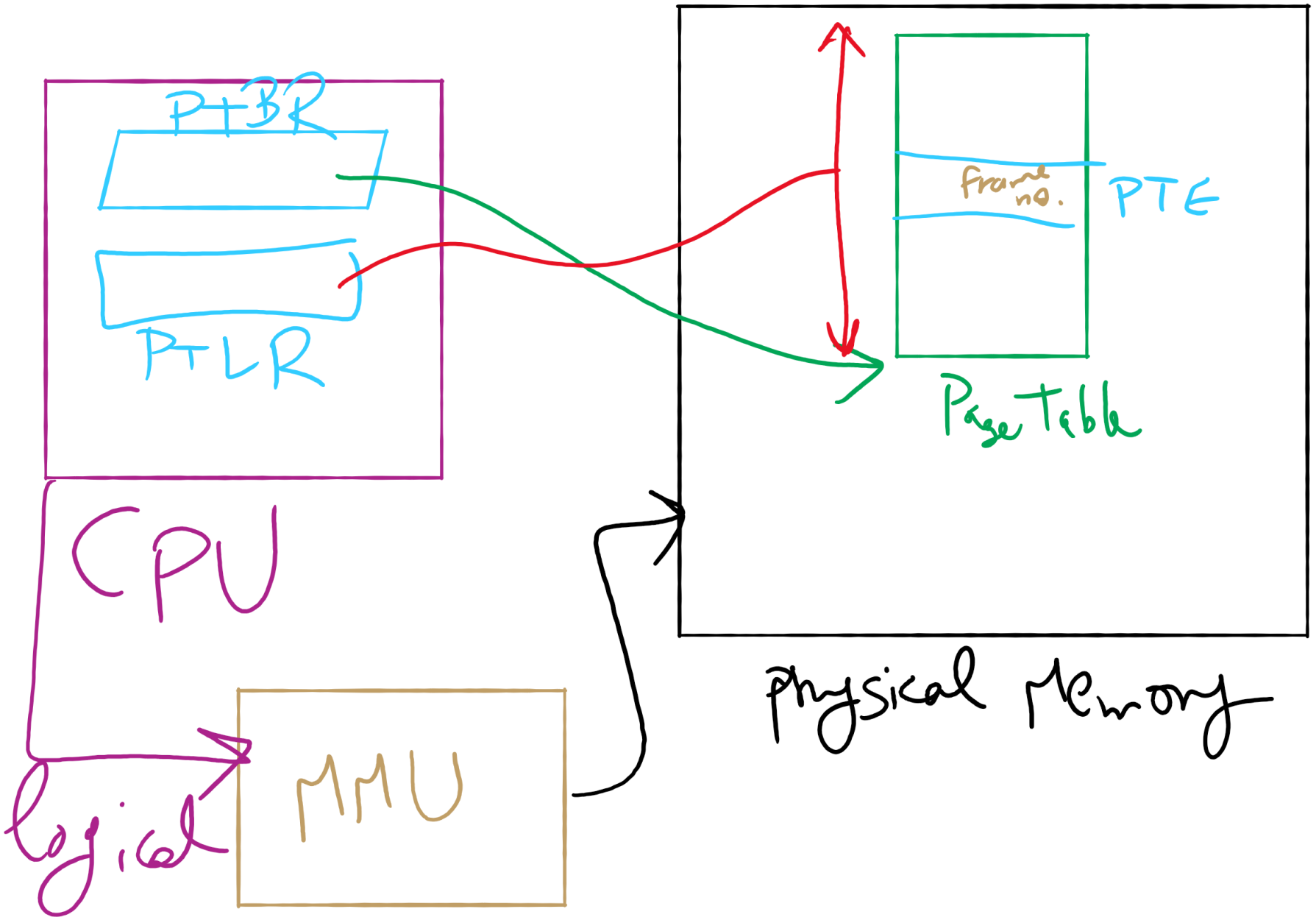
Muddiest Points (2/2)

- In the OPT simulation, why do we only re order the pages above the one we want? Wouldn't it be better to re order everything?
- OPT trace was slightly confusing of what switches where
- How to find the number of writes for LRU and OPT?
- How to calculate the distance in the tables we made
- Are local or global selection policies more common in modern OS's?

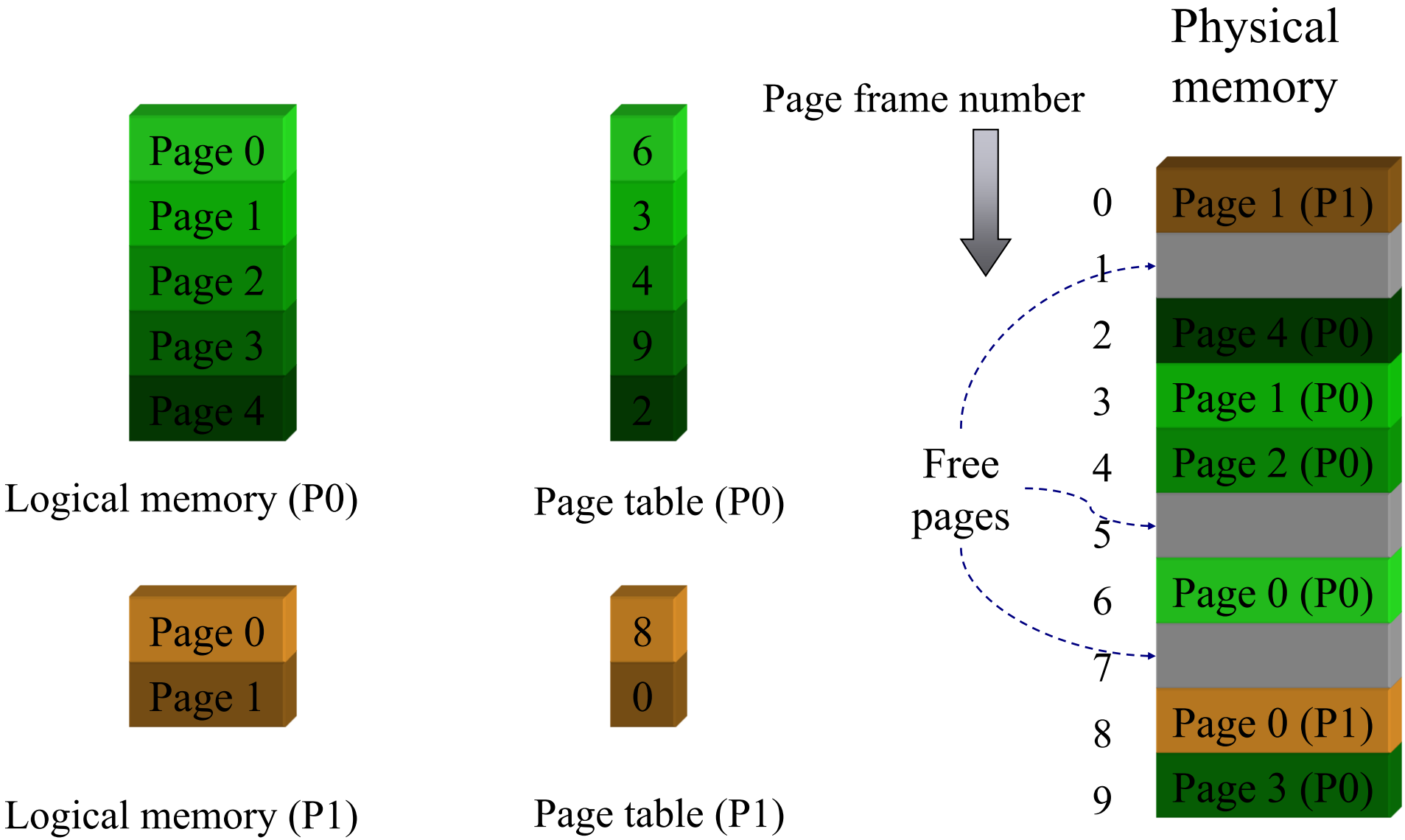
Problem of the Day

- How to keep track of larger page tables that can store more pages
- How big can a page table be?
  - 64-bit machine
  - 4 KB page size
  - How many pages?
  - How many PTE?
  - How big is a PTE?
  - How big is the page table of one process?

Address Translation Structures



Memory & paging structures

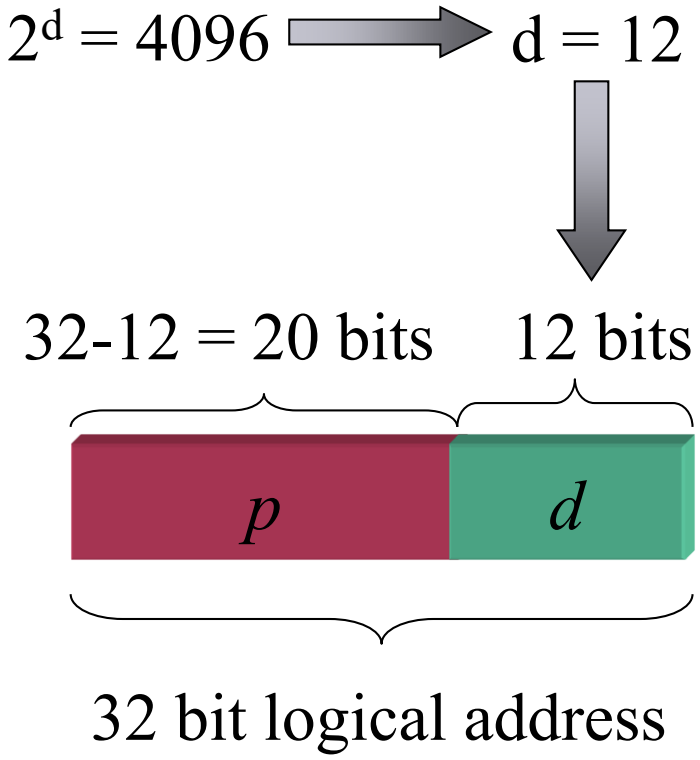


Mapping logical => physical address

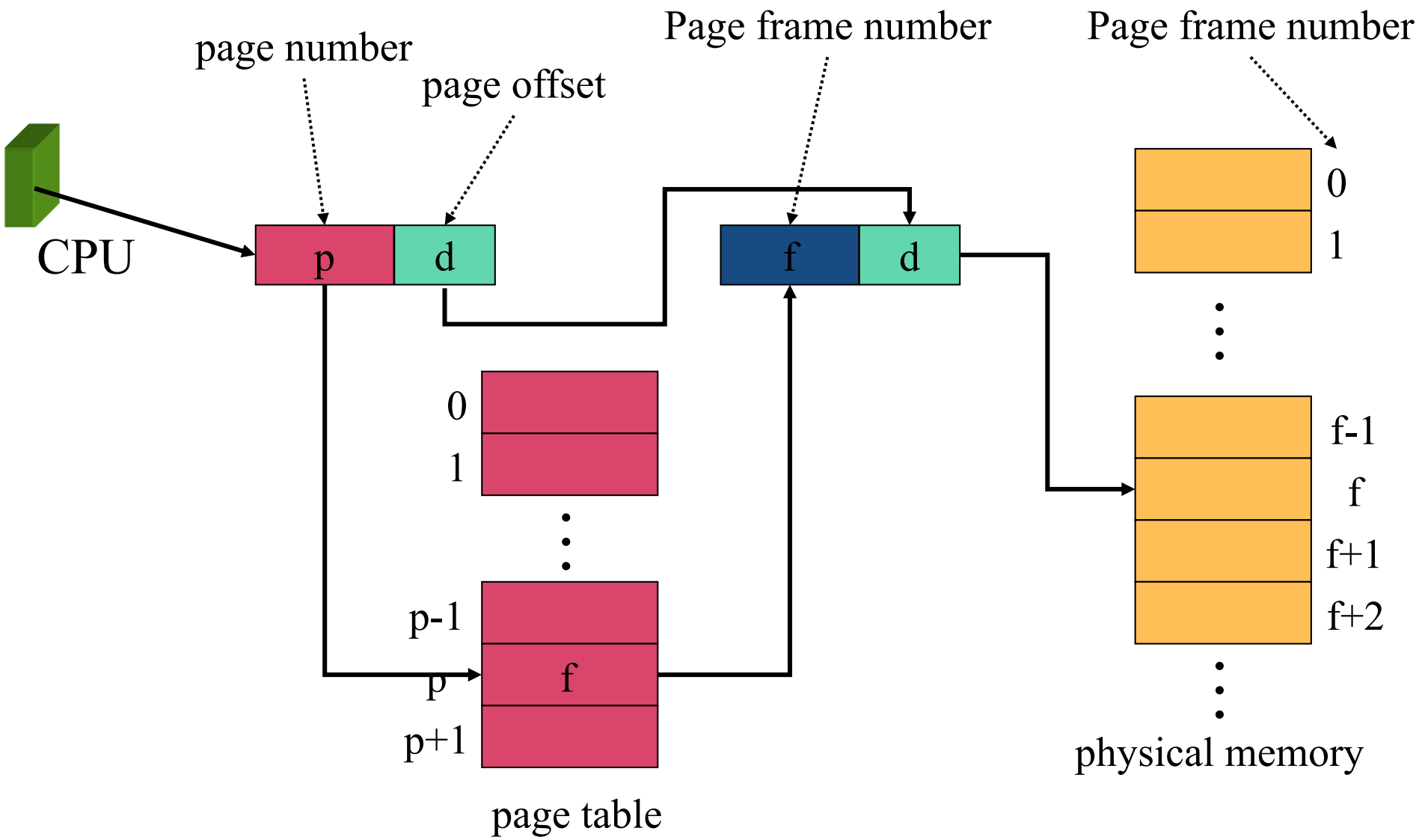
- Split address from CPU into two pieces
  - Page number (*p*)
  - Page offset (*d*)
- Page number
  - Index into page table
  - Page table contains base address of page in physical memory
- Page offset
  - Added to base address to get actual physical memory address
- Page size = 2^d bytes

Example:

- 4 KB (=4096 byte) pages
- 32 bit logical addresses



Address translation architecture



Memory Sizes

KiB = 2^{10}

MiB = 2^{20}

GiB = 2^{30}

TiB = 2^{40}

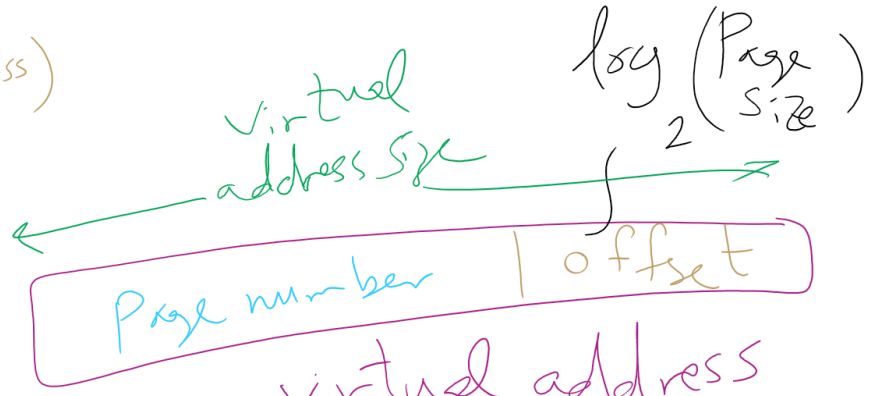
PiB = 2^{50}

Address Translation Relations

$$\#PTEs = \frac{\text{Virtual address space size}}{\text{Page size}} = \frac{2^{(\text{logical address size})}}{\text{Page size}}$$

$$= 2^{(\text{Page no. size})}$$

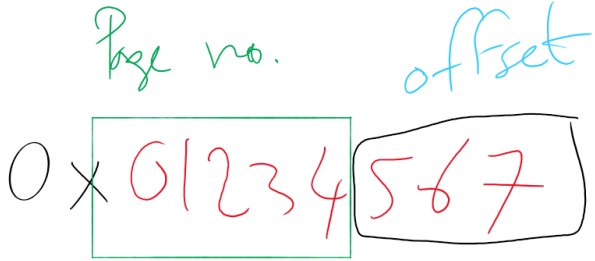
$$\text{Page Table size} = \#PTEs * PTE\ size$$





$$\text{frame number size} = \lceil \log_2 \#frames \rceil$$

$$\#frames = \frac{RAM\ size}{frame\ size}$$

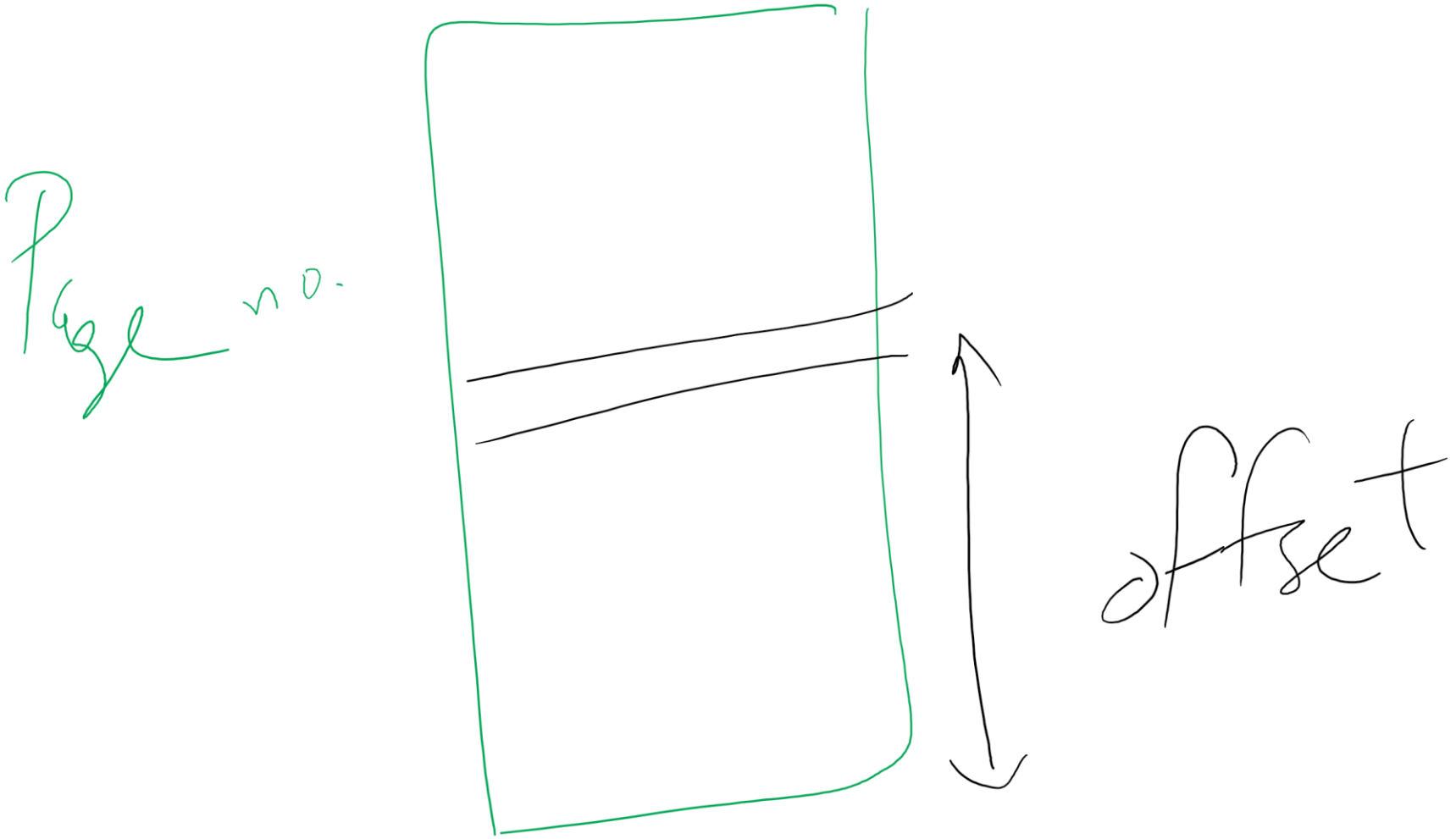


Page size = 4 KB = 2^{12} bytes

offset : 12 bits

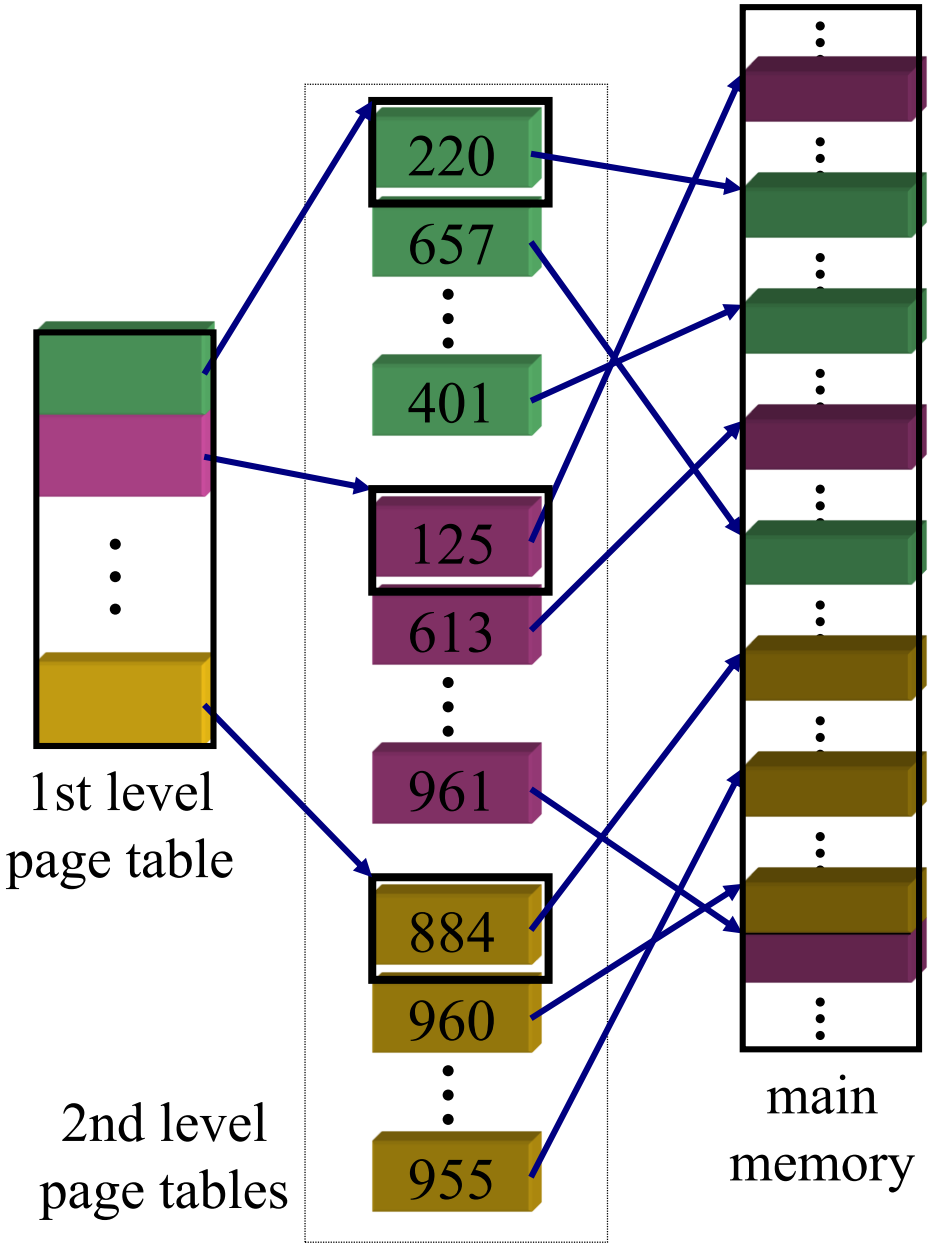
: 3 hexadecimal digits

Page number vs. Offset

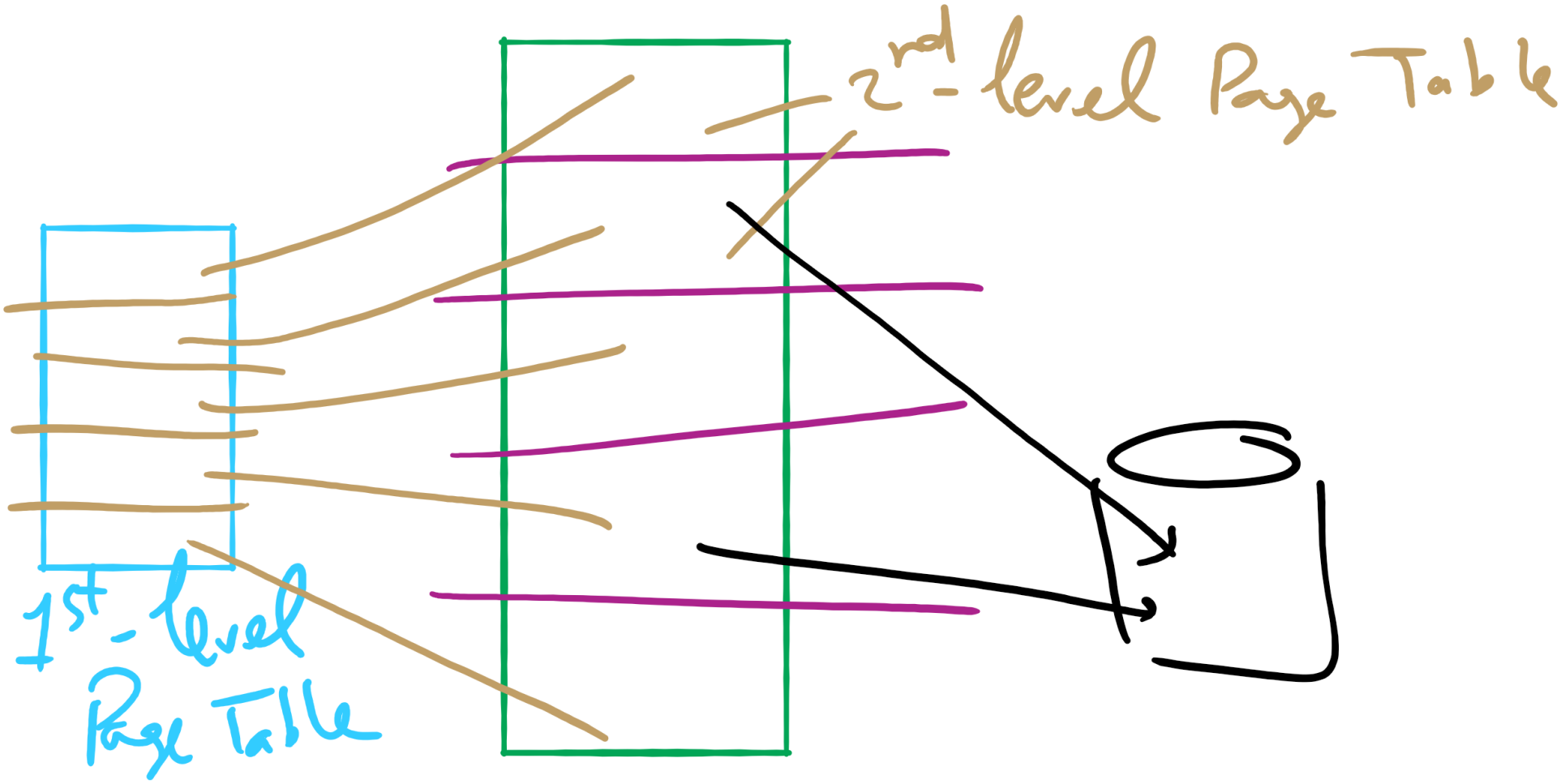


Solution 1: Two-level page tables

- Problem: page tables can be too large
  - 2^{32} bytes in 4KB pages need 1 million PTEs
- Solution: use multi-level page tables
  - "Page size" in first page table is large (megabytes)
  - PTE marked invalid in first page table needs no 2nd level page table
- 1st level page table has pointers to 2nd level page tables
- 2nd level page table has actual physical page numbers in it



Two-level Page Table

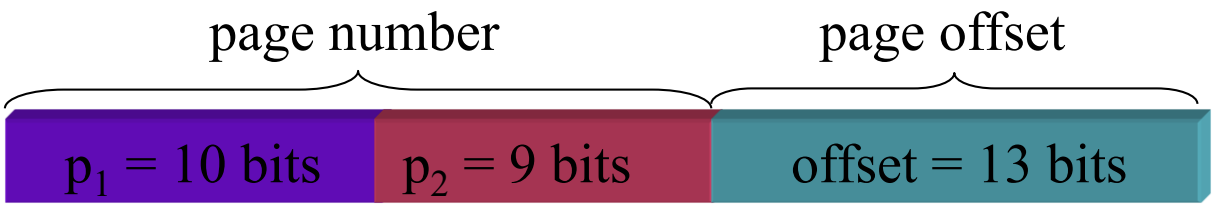


More on two-level page tables

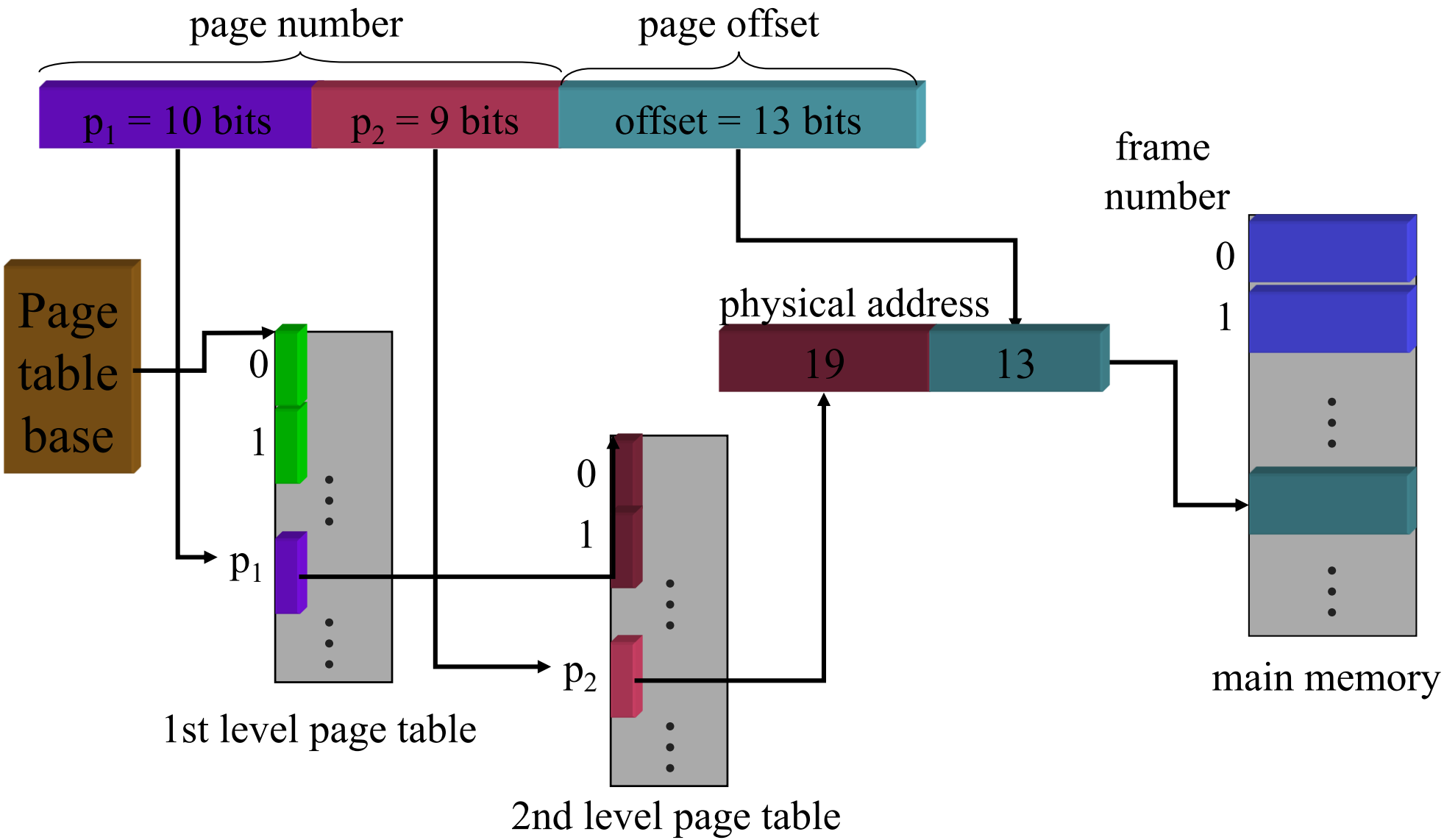
- Tradeoffs between 1st and 2nd level page table sizes
  - Total number of bits indexing 1st and 2nd level table is constant for a given page size and logical address length
  - Tradeoff between number of bits indexing 1st and number indexing 2nd level tables
    - More bits in 1st level: fine granularity at 2nd level
    - Fewer bits in 1st level: maybe less wasted space?
- All addresses in table are physical addresses
- Protection bits kept in 2nd level table

Two-level paging: example

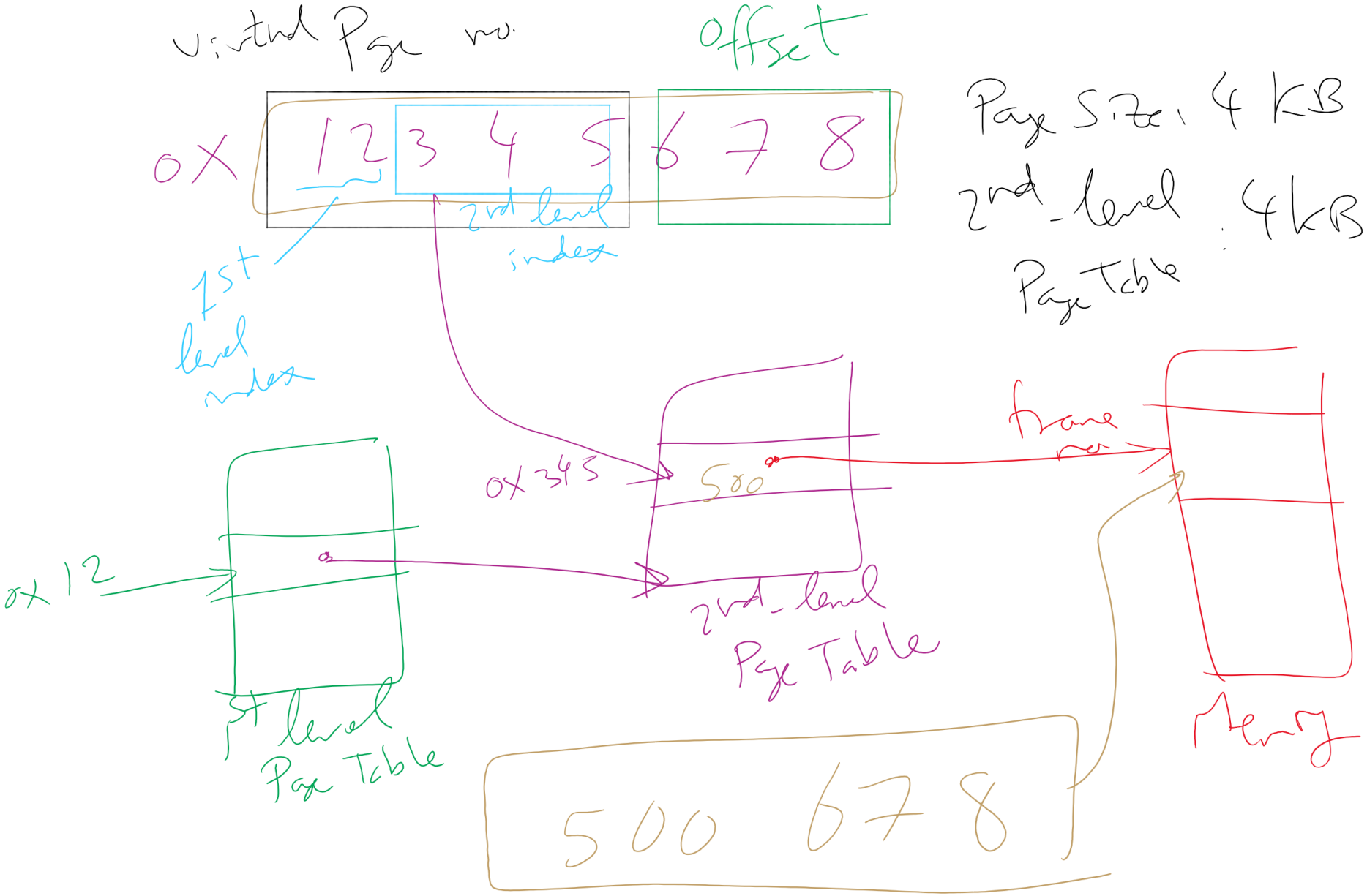
- System characteristics
  - 8 KB pages
  - 32-bit logical address divided into 13 bit page offset, 19 bit page number
- Page number divided into:
  - 10 bit page number
  - 9 bit page offset
- Logical address looks like this:
  - p_1 is an index into the 1st level page table
  - p_2 is an index into the 2nd level page table pointed to by p_1



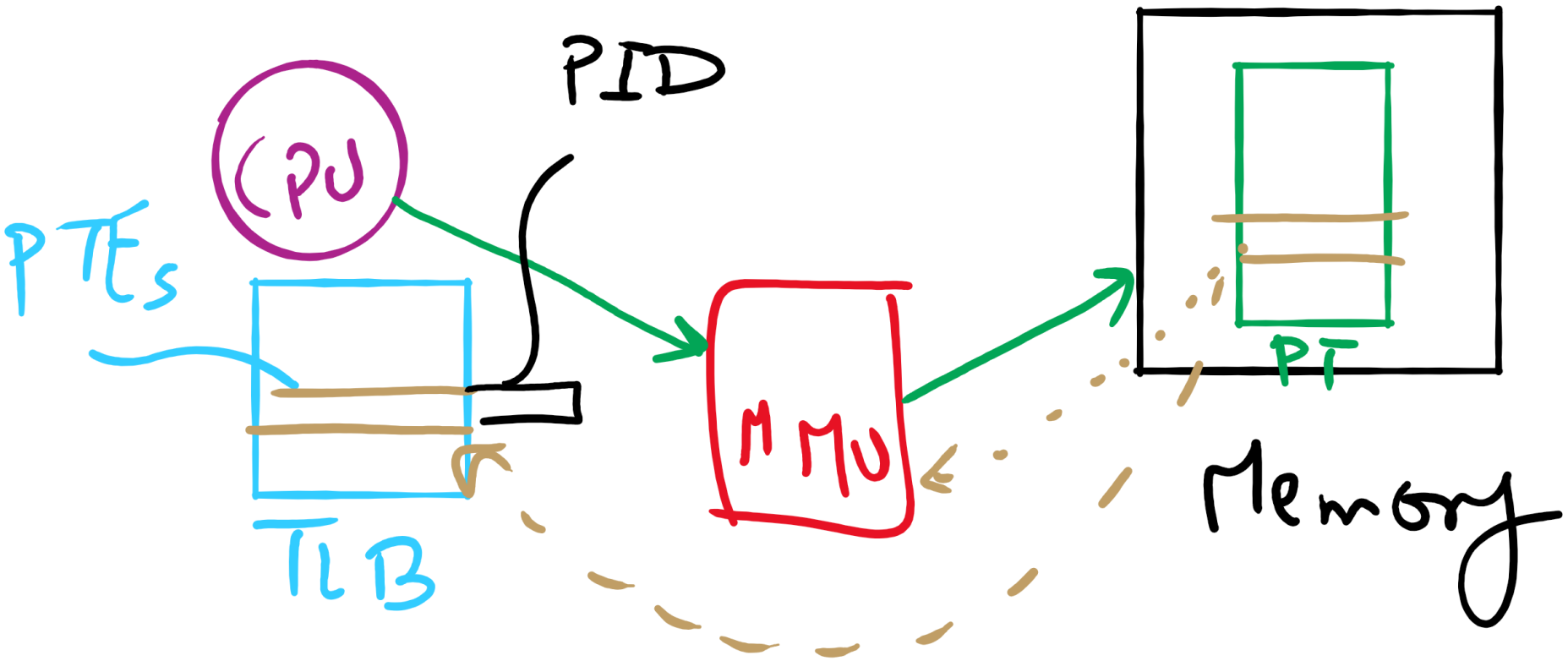
2-level address translation example



Address Translation: 2-level Page Table

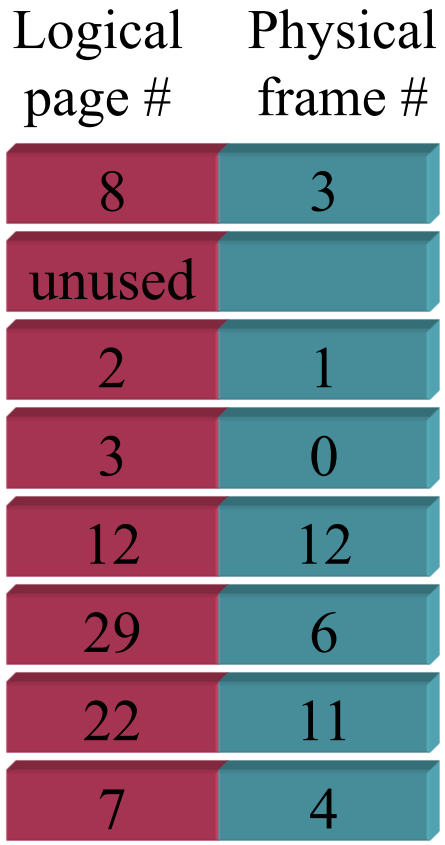


TLB



Translation Lookaside Buffer (TLB)

- Search the TLB for the desired logical page number
  - Search entries in parallel
  - Use standard cache techniques
- If desired logical page number is found, get frame number from TLB
- If desired logical page number isn't found
  - Get frame number from page table in memory
  - Replace an entry in the TLB with the logical & physical page numbers from this reference



Example TLB

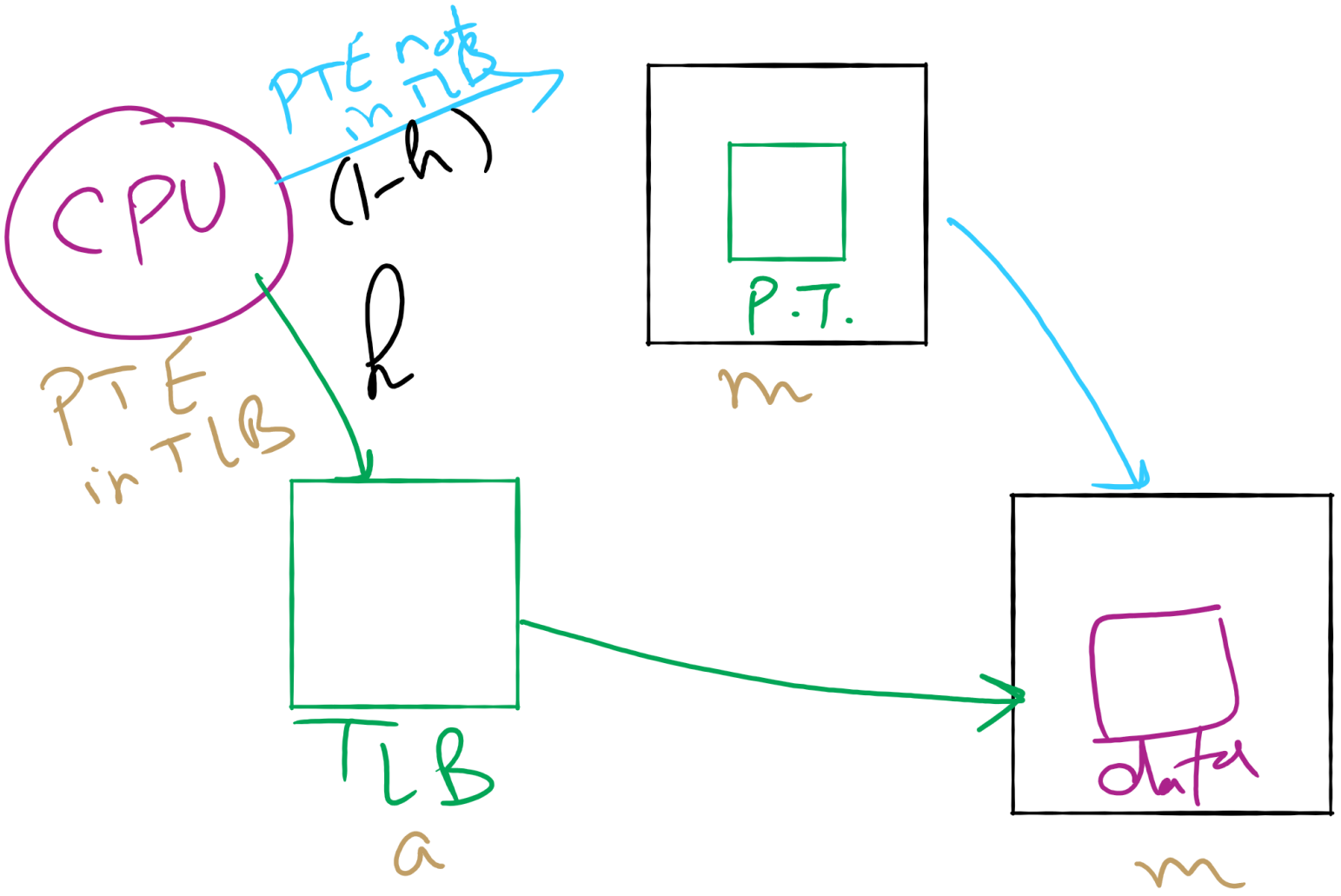
Handling TLB misses

- If PTE isn't found in TLB, OS needs to do the lookup in the page table
- Lookup can be done in hardware or software
- Hardware TLB replacement
  - CPU hardware does page table lookup
  - Can be faster than software
  - Less flexible than software, and more complex hardware
- Software TLB replacement
  - OS gets TLB exception
  - Exception handler does page table lookup & places the result into the TLB
  - Program continues after return from exception
  - Larger TLB (lower miss rate) can make this feasible

How long do memory accesses take?

- Assume the following times:
  - TLB lookup time = a (often zero - overlapped in CPU)
  - Memory access time = m
- Hit ratio (h) is percentage of time that a logical page number is found in the TLB
  - Larger TLB usually means higher h
  - TLB structure can affect h as well
- Effective access time (an average) is calculated as:
  - EAT = (m + a)h + (m + m + a)(1-h)
  - EAT =a + (2-h)m
- Interpretation
  - Reference always requires TLB lookup, 1 memory access
  - TLB misses also require an additional memory reference

Effective Access Time



$$EAT = h(a+m) + (1-h)(a+m+m)$$

EAT Calculation

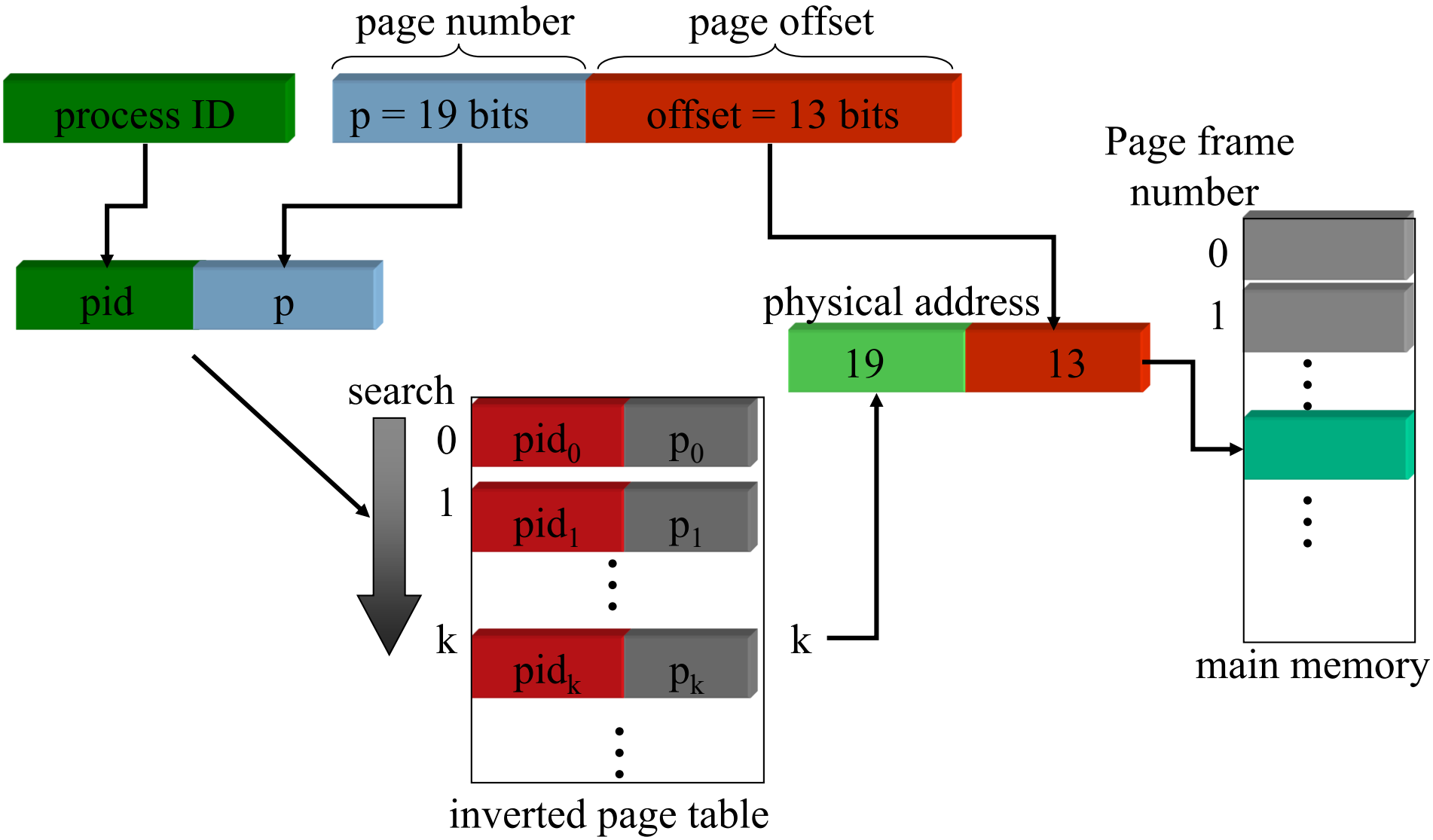
$$EAT = h(a+m) + (1-h)(a+m+m)$$

2-level Page Table → $2m$

Solution 2: Inverted page table

- Reduce page table size further: keep one entry for each frame in memory
- PTE contains
  - Virtual address pointing to this frame
  - Information about the process that owns this page
- Search page table by
  - Hashing the virtual page number and process ID
  - Starting at the entry corresponding to the hash result
  - Search until either the entry is found or a limit is reached
- Page frame number is index of PTE
- Improve performance by using more advanced hashing algorithms

Inverted page table architecture



Inverted Page Table Example

